Structured programming -lab

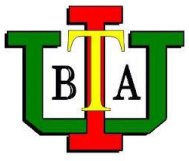
Course code: cse 1383
Assignment project
Title: medicine stock and expiry management for a community health center

Submitted to:

Abdullah Mohammad sakib
lecturer
Department of computer science and technology

Submitted by:

Tawhidul islam
Id:26103042
Program: BCSE
Section: D
Serial :9
Submission: 6-9-2026



IUBAT- international university of business agricultural and technology

1. Introduction

1.1 Background

A community health center depends on the continuous and reliable availability of essential medicines to serve the patients who visit it. The quantity of any given medicine held in store is not a static figure; it changes on a near-daily basis as a result of ongoing patient consumption, occasional bulk issues, and periodic replenishment from suppliers. Because the center typically operates with limited storage space, a limited procurement budget and a limited window in which newly received stock can be expected to arrive, the way in which the available quantity of every medicine is monitored has a direct and material bearing on the quality of care that the center is able to provide.

Two opposing failure modes characterize poor stock management in this setting. On one hand, an excessively cautious approach that keeps very large quantities of every medicine in store reduces the likelihood of an outright shortage, but increases the likelihood that a portion of that stock will remain unused when its expiry period is reached, resulting in wastage of medicine, of the funds spent to procure it, and of the storage capacity it occupied. On the other hand, an excessively lean approach that keeps only small quantities in store reduces the likelihood of such wastage, but increases the likelihood that a sudden increase in patient demand, or a delay in the next scheduled supply, will leave the center without a medicine that a patient urgently requires.

The problem is further complicated by the fact that not every medicine carries the same clinical weight. A medicine that is essential to patient care, and for which no substitute is readily available, must not be allowed to fall into shortage even briefly, whereas a temporary and brief shortage of a comparatively routine medicine may be tolerated with far less consequence. Similarly, a medicine that currently shows a comfortable stock level may still deserve managerial attention if a substantial share of that stock is approaching the end of its usable shelf life and is unlikely to be consumed in time.

1.2 Objective of the Developed Program

The objective of this assignment is to design and implement, in the C programming language, a structured software solution that examines the current medicine stock of a community health center, identifies medicines exposed to shortage risk, expiry risk, or a combination of both, determines an appropriate order in which the medicines should receive managerial attention, and produces an organized, file-recorded report that the management of the center can act upon. The program is required to operate correctly not only on the six medicines supplied for the purposes of this assignment, but on any comparable dataset of medicine records, since the values used for demonstration are illustrative rather than fixed.

2. Problem Analysis

The dataset supplied for the purpose of this assignment, and used throughout this report for illustration, is reproduced below.

Medicine Code	Medicine Name	Available Stock	Avg. Daily Requirement	Minimum Stock Level	Days Remaining to Expiry	Essentiality Level
MED01	Oral Saline	120	35	80	45	3
MED02	Paracetamol	300	40	100	120	2
MED03	Insulin	60	12	50	30	3
MED04	Amoxicillin	200	25	80	20	3
MED05	Antacid	250	18	60	15	1
MED06	Cetirizine	180	15	50	90	1

2.1 Major Requirements

- Represent, for each medicine, its code, name, available stock, average daily requirement, minimum stock level, days remaining to expiry and essentiality level, in a manner that keeps these seven attributes correctly associated with one another under every operation performed by the program.
- Derive, from the stored attributes, further quantities that are useful for decision-making but are not themselves supplied directly, most notably the number of days for which the present stock is expected to remain available under the recorded average demand.
- Classify every medicine into a stock condition that reflects its actual situation, rather than a single generic status.
- Rank the medicines in an order that reflects the urgency with which each one requires managerial attention, and resolve ties between medicines of equal computed urgency in a defensible manner.
- Provide a facility through which a member of staff can look up a single medicine by an identifying value and see its full current status.
- Allow the available stock of a medicine to be corrected upward, on receipt of new supply, or downward, on issue to a patient, without ever allowing the recorded stock to become negative, and ensure that the medicine's derived status is updated immediately afterwards.
- Produce a final, organized report, together with a summary of how many medicines fall into each category of concern, and preserve this report in a file for later reference.

2.2 Stock-Related Constraints

The available stock of every medicine must be compared against two distinct reference points: the minimum stock level, which represents the threshold below which the centre considers the medicine to be under-stocked, and the average daily requirement, which represents the rate at which the stock is actually consumed. A medicine can be below its minimum stock level and yet still have several days of coverage remaining if its daily requirement is modest; conversely, a medicine can be above its minimum stock level and yet face a genuine shortage risk within a short time if its daily requirement is comparatively high. Both figures must therefore be considered together rather than in isolation, and the analysis must not rely on the available stock figure alone.

2.3 Expiry-Related Constraints

The days remaining to expiry indicates how much time is left before a medicine's current batch can no longer be safely dispensed. A large available stock does not, by itself, indicate a healthy stock position if the remaining shelf life is short and the recorded daily requirement is not high enough to consume the stock before that period elapses. The program must therefore estimate, for every medicine, whether the quantity currently held is realistically expected to be consumed within the remaining expiry period, and treat any portion that is not expected to be consumed in time as being at risk of wastage, independently of whether the same medicine is also short of its minimum stock level.

2.4 Conflicting Requirements

Several of the requirements identified above pull the proposed solution in opposing directions, and the engineering considerations stated in the assignment brief must be reconciled rather than resolved in favor of one side alone. These conflicting pairs are summarized below.

Requirement	Conflicting Requirement
Maintain sufficient medicine stock	Avoid excessive stock and expiry-related wastage
Replenish low-stock medicine	Avoid unnecessary replenishment when existing stock may expire
Give priority to essential medicine	Attend to other medicines having immediate expiry risk
Maintain a simple decision method	Consider sufficient information for a reliable decision
Process stock information accurately	Avoid unnecessary repeated calculations and processing

The proposed solution, described in the following section, is designed to hold a reasoned balance between each of these opposing pairs rather than to optimize any single one of them in isolation.

3. Proposed Solution

3.1 Method Used to Analyze Stock

For every medicine, the program derives an estimated stock-coverage period, expressed in days, by dividing the available stock by the average daily requirement. This single derived quantity is central to the remainder of the analysis, since it converts an absolute stock figure into a time-based measure that can be meaningfully compared against the minimum stock level, expressed as an equivalent coverage period, and against the days remaining to expiry, which is already expressed in the same unit. The stock-coverage period is calculated once for a given set of recorded values and reused wherever it is needed afterwards, rather than being recalculated repeatedly, in keeping with the optimization requirement of the assignment.

In addition to the stock-coverage period, the program estimates the quantity of a medicine that is realistically expected to be consumed before its recorded expiry period is reached, by multiplying the average daily requirement by the days remaining to expiry. Comparing this expected consumption against the available stock allows the program to determine, for every medicine, whether any portion of the

current stock is likely to remain unused when it expires, which is the basis of the expiry-related decision described in the next subsection.

3.2 Method Used to Identify Stock Condition

Rather than relying on the available stock figure in isolation, the proposed classification considers, jointly, the relationship of the available stock to the minimum stock level, the estimated stock-coverage period relative to the essentiality level of the medicine, and the estimated expiry exposure derived above. On this combined basis, each medicine is assigned to one of the following five conditions.

Stock Condition	Governing Situation (Informal Description)
Sufficient Stock	Available stock is at or above the minimum stock level, the estimated stock-coverage period comfortably exceeds the time needed for the next replenishment cycle, and no significant expiry concern exists.
Reorder Required	Available stock has fallen below the minimum stock level, or the estimated stock-coverage period is short, but the medicine is not of the highest essentiality and no immediate patient-care risk is indicated.
Urgent Reorder	Available stock is below the minimum stock level for a medicine of high essentiality, or the estimated stock-coverage period is shorter than the time realistically required to obtain fresh supply.
Expiry Attention	Available stock is sufficient (at or above the minimum stock level) but a considerable portion of that stock is not expected to be consumed before the recorded expiry period, indicating a risk of wastage.
Critical Condition	The medicine simultaneously exhibits shortage risk (low stock relative to demand) and expiry risk (short remaining shelf life), or is an essential medicine whose coverage period is shorter than its remaining expiry period, requiring combined and immediate managerial decision-making.

The distinguishing feature of this classification, compared with a scheme based on the available stock alone, is that it separates a medicine that is genuinely short of stock from a medicine that merely appears well-stocked but is exposed to wastage, and it additionally recognizes that a medicine can display both concerns at once, in which case it is placed in the Critical Condition category rather than under either concern alone.

3.3 Method Used to Determine Management Priority

Because the center cannot address every outstanding stock concern simultaneously, the medicines must be placed in an order that reflects the urgency of managerial attention each one requires. The proposed method combines the factors listed below into a single, comparable priority indicator for every medicine, rather than ranking the medicines on any one factor alone.

Factor	Symbol	Interpretation in the Priority Score
Essentiality Level	E	Higher essentiality contributes a larger share of the priority score, since a shortage of an essential medicine has more serious consequences for patient care.

Factor	Symbol	Interpretation in the Priority Score
Shortage Severity	S	Computed from how far the available stock has fallen below the minimum stock level, expressed as a proportion; a medicine already below its minimum level contributes more strongly than one merely approaching it.
Stock Coverage (days)	C	The number of days the current stock is expected to last under average daily consumption; a shorter coverage period increases urgency.
Expiry Urgency	X	Derived from the days remaining to expiry; a medicine nearing expiry, particularly one that also shows shortage risk, contributes a higher urgency component.
Combined Risk Indicator	R	A supplementary indicator that increases the score further when both shortage risk and expiry risk are present simultaneously, reflecting the added managerial complexity of such cases.

A medicine of high essentiality that is already below its minimum stock level, and whose estimated coverage period is short, receives the highest combined priority indicator and is therefore placed at the top of the management-attention order. A medicine that shows only a mild expiry exposure, with sufficient stock and lower essentiality, receives a comparatively low priority indicator and is placed toward the bottom of the order. Because the priority indicator for a given medicine depends only on values that have already been derived while analyzing that medicine's stock condition, it is computed directly from those stored, previously calculated results rather than by re-examining the raw data a second time, again in keeping with the optimization requirement.

3.4 Method Used to Resolve Equal-Priority Cases

Where two or more medicines produce an identical, or near-identical, combined priority indicator, the proposed solution resolves the tie through a short sequence of secondary comparisons applied in order: first, the medicine with the higher essentiality level is placed ahead; second, if the essentiality levels are also equal, the medicine with the shorter estimated stock-coverage period is placed ahead, since it faces the more immediate risk of running out; and third, if the coverage periods are also equal, the medicine with fewer days remaining to expiry is placed ahead, since its exposure window is narrower. This sequence of tie-breaking comparisons is deterministic, requires no additional data beyond what has already been derived, and reflects the same clinical reasoning applied to the primary ranking itself.

3.5 Handling Stock and Expiry Conflict

The assignment brief specifically requires that the proposed solution not automatically recommend additional stock for every medicine that appears low, without regard to whether the same or another medicine is simultaneously carrying a quantity that will expire shortly. The proposed solution addresses this by keeping the shortage indicator and the expiry indicator described above logically separate from one another throughout the analysis, rather than merging them into a single undifferentiated flag. A medicine is reported as facing shortage risk only on the basis of its stock-coverage period relative to its minimum stock level; it is reported as facing expiry risk only on the basis of its expected consumption relative to its recorded expiry period; and it is reported as facing combined risk only when both conditions are met at once. This separation allows the final report to state plainly, for each medicine, which of the two concerns applies, so that the center’s management is not misled into ordering fresh stock of a

medicine whose real difficulty is that existing stock is about to expire unused, nor into overlooking a medicine that is both short of stock and carrying batches close to expiry, which represents the most demanding case for managerial decision-making.

4. Program Design

4.1 Data Organization

Each medicine is represented as a single structured record containing its code, name, available stock, average daily requirement, minimum stock level, days remaining to expiry and essentiality level as supplied, together with derived fields for the estimated stock-coverage period, the estimated expiry exposure, the assigned stock condition and the computed priority indicator. The complete set of medicine records is held in a single array of such structures, sized to accommodate the supplied dataset while remaining straightforward to extend if the number of medicines changes, in line with the requirement that the program continue to operate correctly when the supplied values are changed. Holding the derived fields alongside the supplied fields, within the same record, avoids the need for separate parallel arrays and ensures that every value associated with a given medicine moves together whenever the array is rearranged, searched or updated.

4.2 Major Functions

The proposed solution is organized into a small number of clearly separated functions, each responsible for a single aspect of the overall task, so that repeated operations are implemented once and reused wherever they are required. The intended responsibility of each function is summarized below; the corresponding source code is presented separately as part of the submitted implementation.

Proposed Function (Purpose)	**Responsibility**
Data initialization routine	Loads the medicine records (code, name, available stock, average daily requirement, minimum stock level, days remaining to expiry, essentiality level) into the chosen data structure at program start-up, from either in-code initial values or an input file, so that the six supplied medicines and any future dataset of different size can be accommodated without changing the program logic.
Stock-coverage calculator	Derives, for a given medicine record, the estimated number of days the available stock will last under the recorded average daily requirement, and stores the result so that it need not be recalculated every time it is referenced elsewhere in the program.
Expiry-exposure calculator	Derives, for a given medicine record, the quantity that is expected to remain unused at the recorded expiry period under the current consumption rate, distinguishing genuine expiry risk from a simple shortage.
Stock-condition classifier	Applies the reasoned decision rules to the stock-coverage and expiry-exposure results, together with the minimum stock level and essentiality level, to assign one of the defined stock-condition categories to the medicine.

Proposed Function (Purpose)	Responsibility
Priority evaluator	Combines essentiality, shortage severity, stock coverage and expiry urgency into a single comparable priority indicator for each medicine, and resolves equal-priority cases using the secondary and tertiary comparison rules described in the Proposed Solution section.
Search routine	Accepts an identifying value (the medicine code, or alternatively the medicine name) from the user and locates the corresponding record, reporting an appropriate message when no match exists.
Ordering routine	Arranges the medicine records according to the selected criterion (management priority, by default) using a sorting technique appropriate to the small, in-memory dataset, while keeping all fields of a record correctly associated with one another during the rearrangement.
Stock-update routine	Adjusts the available stock of a specified medicine, upward on receipt of new stock or downward on issue to patients, rejects any update that would drive the available stock below zero, and triggers recalculation of only the affected medicine's derived values.
Report generator	Produces the final, organized report (per-medicine figures and the summary counts) on screen and writes the corresponding output to a file.
Menu / control routine	Presents the available operations to the user, accepts a selection, and calls the corresponding routine, allowing the program to be exercised repeatedly during demonstration without restarting.

4.3 Algorithm / Pseudocode

The overall control flow of the proposed program, expressed at a level intended to convey the reasoning of the solution rather than the C-language syntax of its implementation, proceeds as follows.

1. Load the supplied medicine records into the array of structures described in Section 5.1.
2. For every record in the array, in a single pass: derive its stock-coverage period from its available stock and average daily requirement; derive its expiry-exposure quantity from its average daily requirement and days remaining to expiry; assign a stock condition using the rules described in Section 4.2; and compute a priority indicator using the method described in Section 4.3. Store each derived value in the record itself so that it is not recalculated in any later step.
3. Present the user with a menu of the operations supported by the program: search for a medicine, update the stock of a medicine, view the medicines ordered by priority (or another selected criterion), and generate the final report.
4. If the user selects the search operation, accept an identifying value, examine the array for a matching record, and display its full stored and derived information, or an appropriate not-found message if no match exists.
5. If the user selects the update operation, accept the medicine to be updated and the nature of the change (receipt of new stock or issue to patients), apply the change to the available stock field while preventing it from becoming negative, and then re-derive only the stock-coverage period,

expiry exposure, stock condition and priority indicator of that single affected record, leaving every other record untouched.

6. If the user selects the ordering operation, arrange the array according to the selected criterion, using the previously derived priority indicator (or another selected field) as the sort key, applying the equal-priority resolution sequence described in Section 4.4 where required, and keeping every field of a record together during any exchange of position.
7. If the user selects the report operation, iterate over the (optionally ordered) array once, writing the medicine code, name, available stock, estimated stock coverage, days remaining to expiry, essentiality level, stock condition and management priority of every record to the screen and to an output file, and accumulate, in the same pass, the summary counts of medicines requiring reorder, requiring urgent attention, and showing expiry concern, together with the identity of the medicine holding the highest management priority.
8. Return to the menu until the user chooses to end the program.

4.4 Flowchart Description

The corresponding flow proceeds, at a high level, from program start, through the one-time loading and analysis of the dataset described in Step 2 above, into a repeating menu loop that offers the search, update, ordering and reporting operations as separate branches. Each branch performs its operation and then returns control to the menu, so that the loading and initial analysis step is executed exactly once, while the operations that depend on it may be exercised repeatedly, including by the course instructor during demonstration, without repeating the initial analysis unnecessarily. The loop terminates only when the user selects the exit option, at which point the program ends.

5. Optimization Considerations

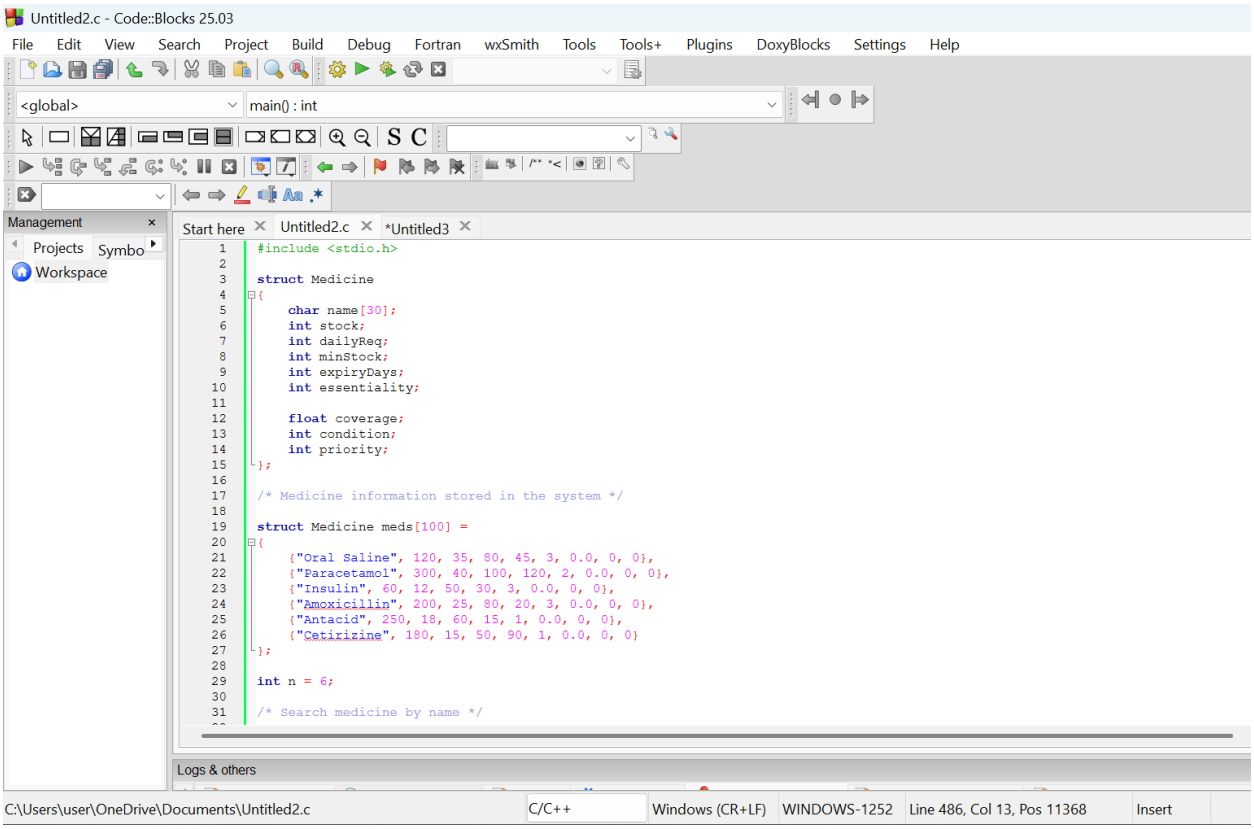
The proposed design applies the optimization requirement of the assignment in the following specific ways.

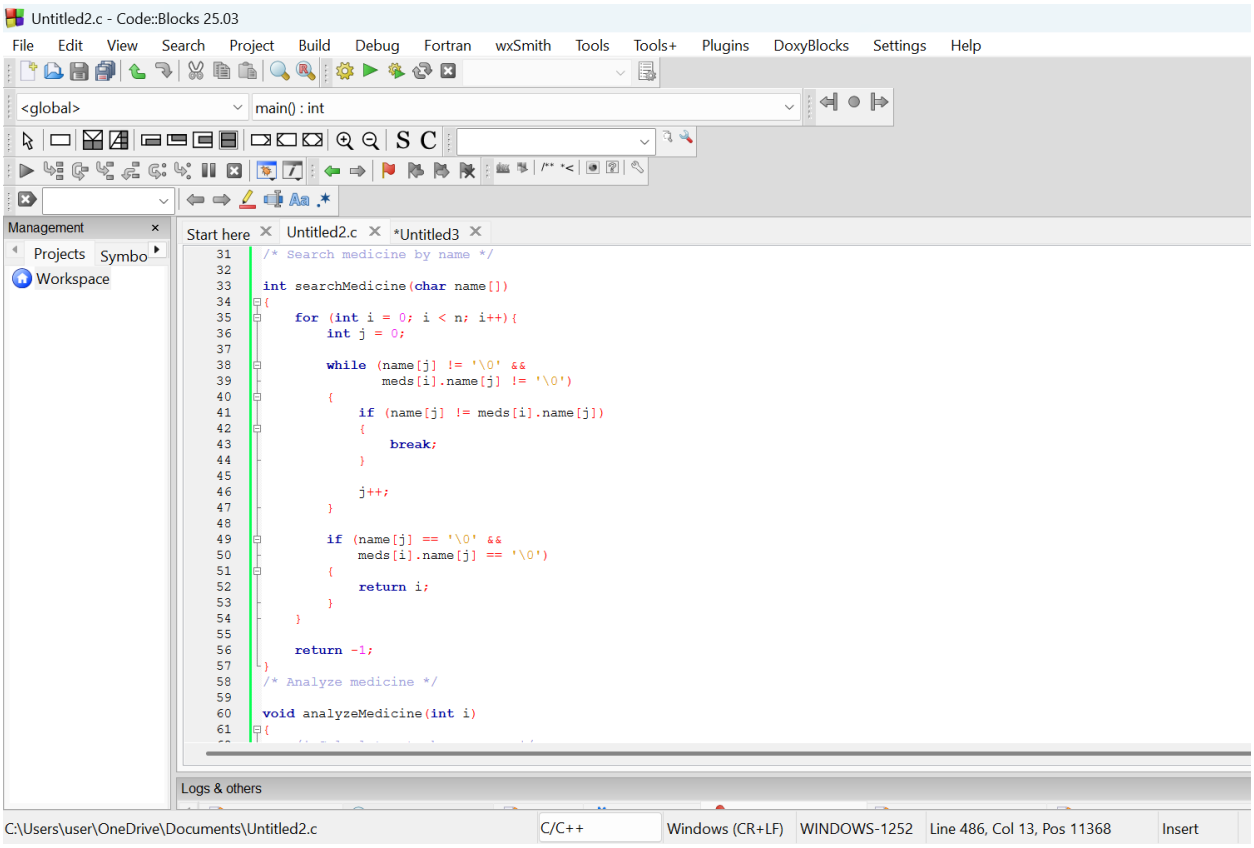
- Single-pass derivation: the stock-coverage period, expiry exposure, stock condition and priority indicator of every medicine are each calculated exactly once, immediately after the data is loaded, and stored within the record itself, rather than being recalculated every time the record is displayed, searched, ordered or reported.
- Localized recalculation on update: when the available stock of a single medicine is changed, only that medicine's derived values are recalculated; the derived values of every other medicine, which are unaffected by the change, are left untouched.
- Reuse of previously identified records: once the search routine has located a medicine, its position in the array is used directly by any subsequent operation requested on that same medicine within the same interaction, rather than searching for it again.
- Consolidated data structure: keeping every attribute of a medicine, supplied and derived, within one structured record removes the need for separate parallel arrays that would otherwise have to be kept synchronized by hand, which would itself be a source of redundant processing and potential inconsistency.
- Single-pass reporting: the summary counts required in the final report (medicines requiring reorder, requiring urgent attention, showing expiry concern, and the medicine of highest priority)

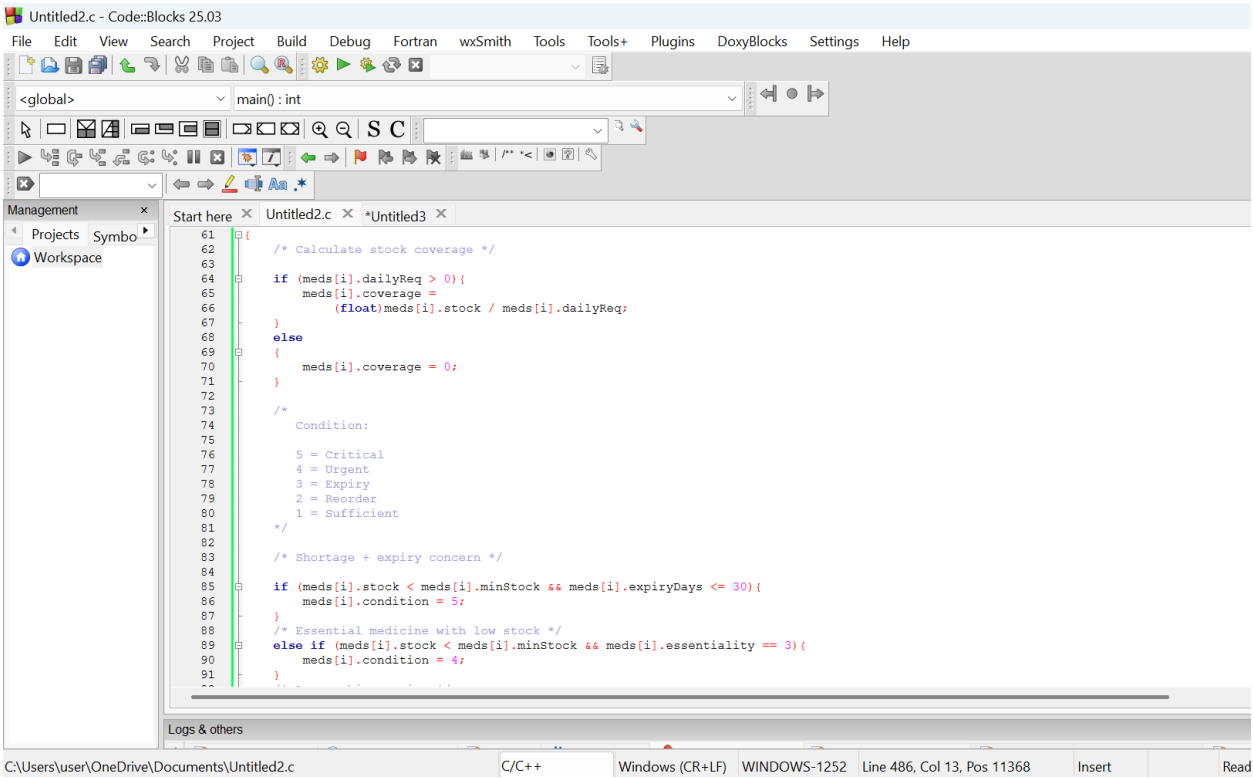
are accumulated during the same pass over the array that produces the per-medicine report lines, rather than in a separate pass.

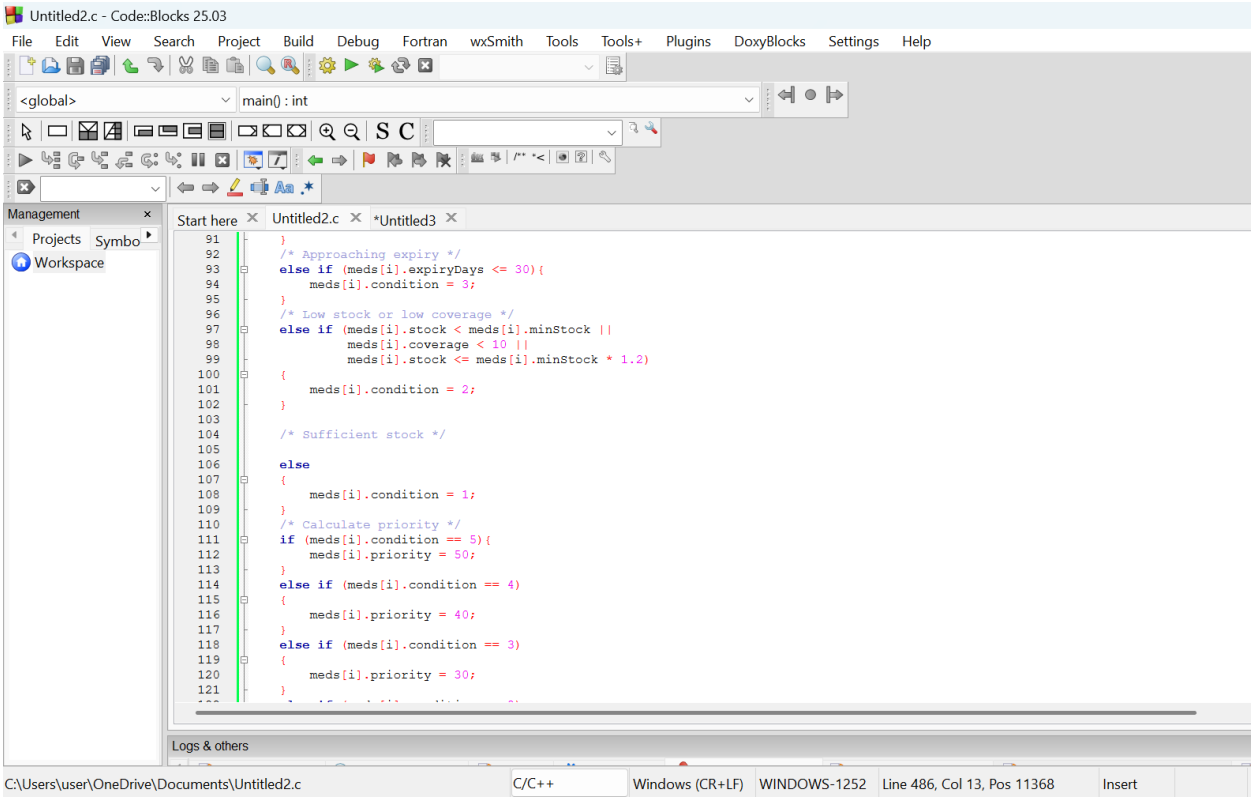
- Selection of a sorting technique proportionate to the dataset: given the small number of medicine records expected in this setting, a straightforward comparison-based sorting technique is considered sufficient and is preferred over a more elaborate technique whose additional complexity would not be justified by the size of the dataset.

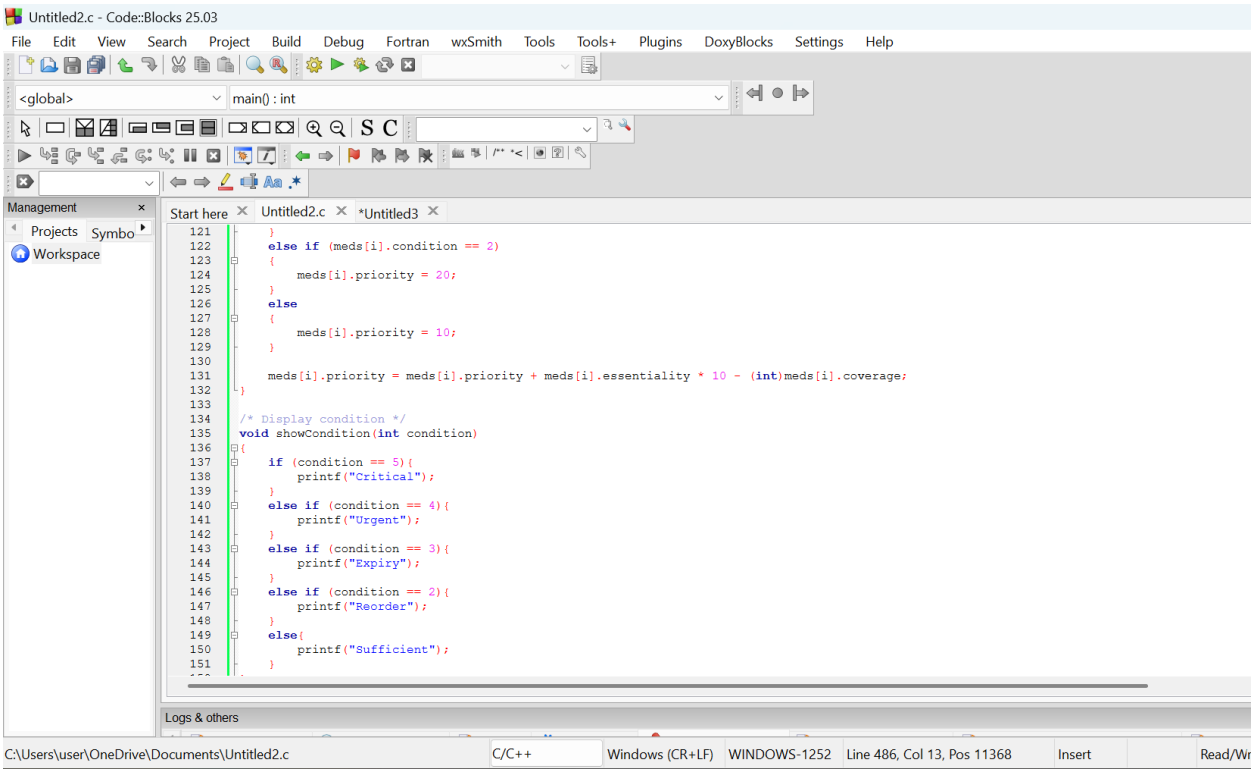
6. code

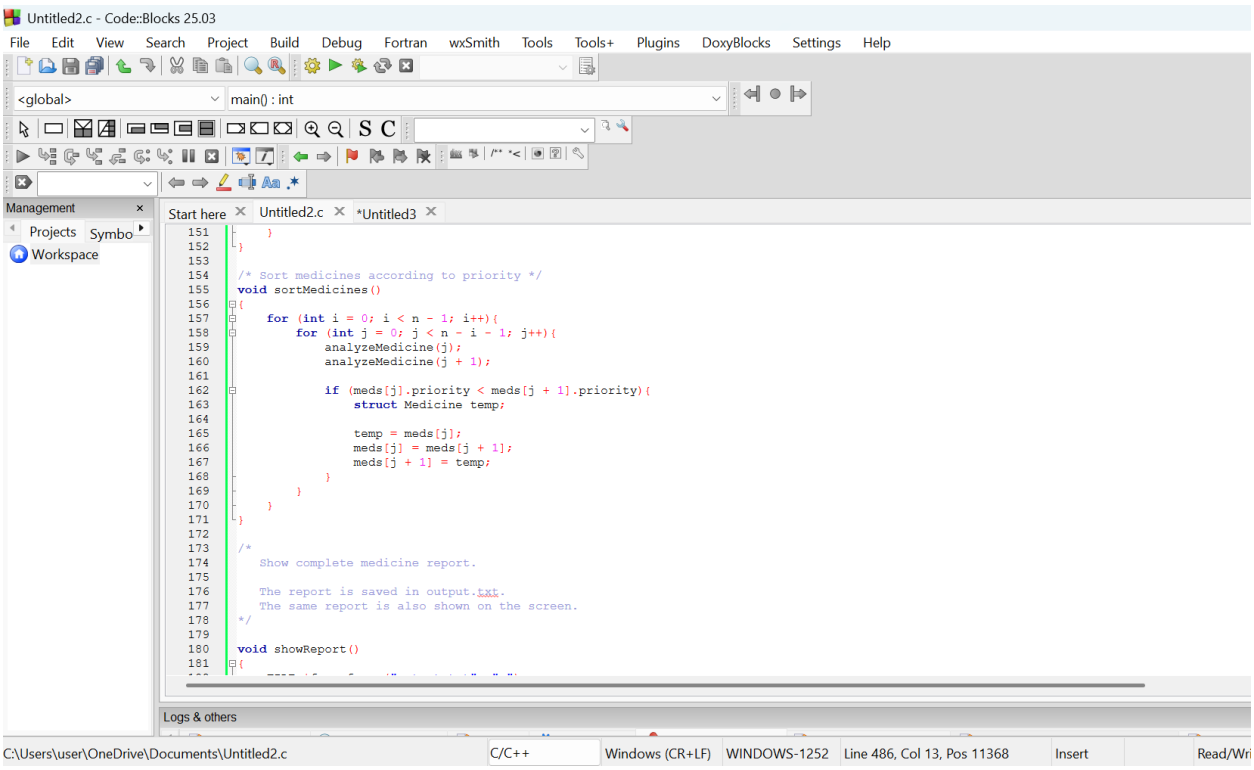


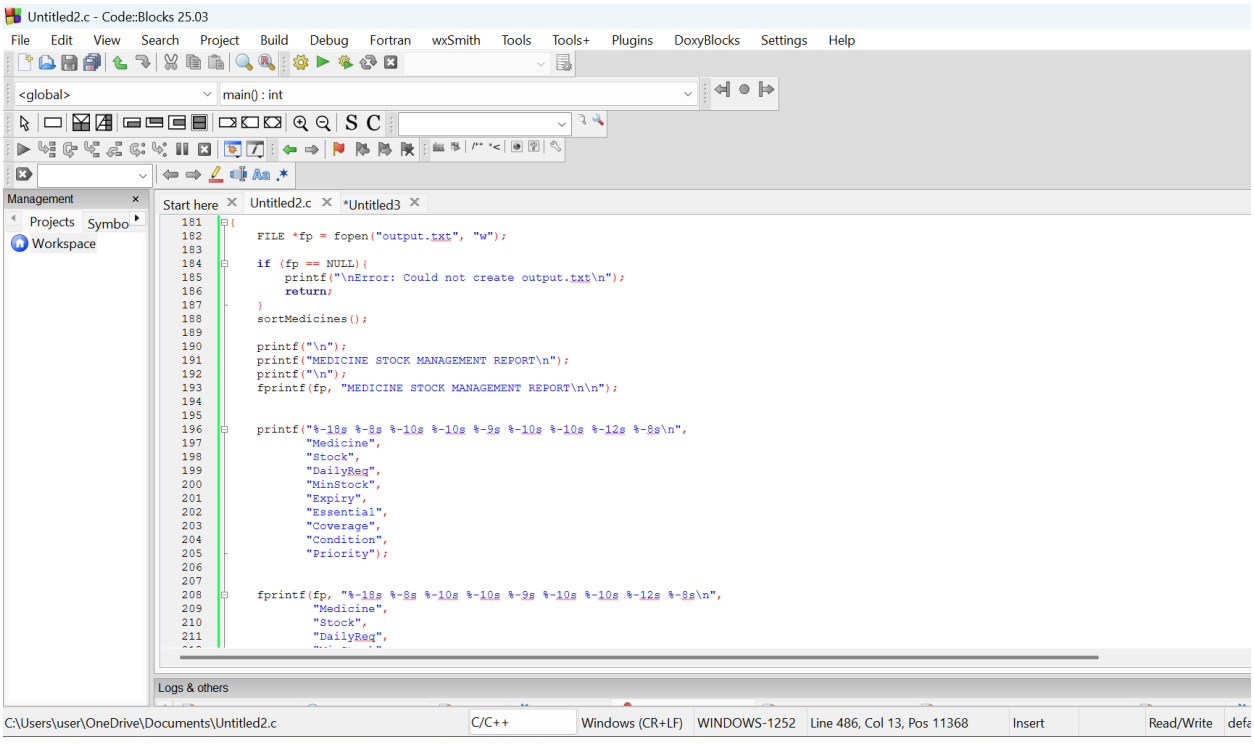

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
31 /* Search medicine by name */
32
33 int searchMedicine(char name[])
34 {
35 for (int i = 0; i < n; i++){
36 int j = 0;
37
38 while (name[j] != '\0' &&
39 meds[i].name[j] != '\0')
40 {
41 if (name[j] != meds[i].name[j])
42 {
43 break;
44 }
45
46 j++;
47 }
48
49 if (name[j] == '\0' &&
50 meds[i].name[j] == '\0')
51 {
52 return i;
53 }
54 }
55
56 return -1;
57 }
58 /* Analyze medicine */
59
60 void analyzeMedicine(int i)
61 {
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert

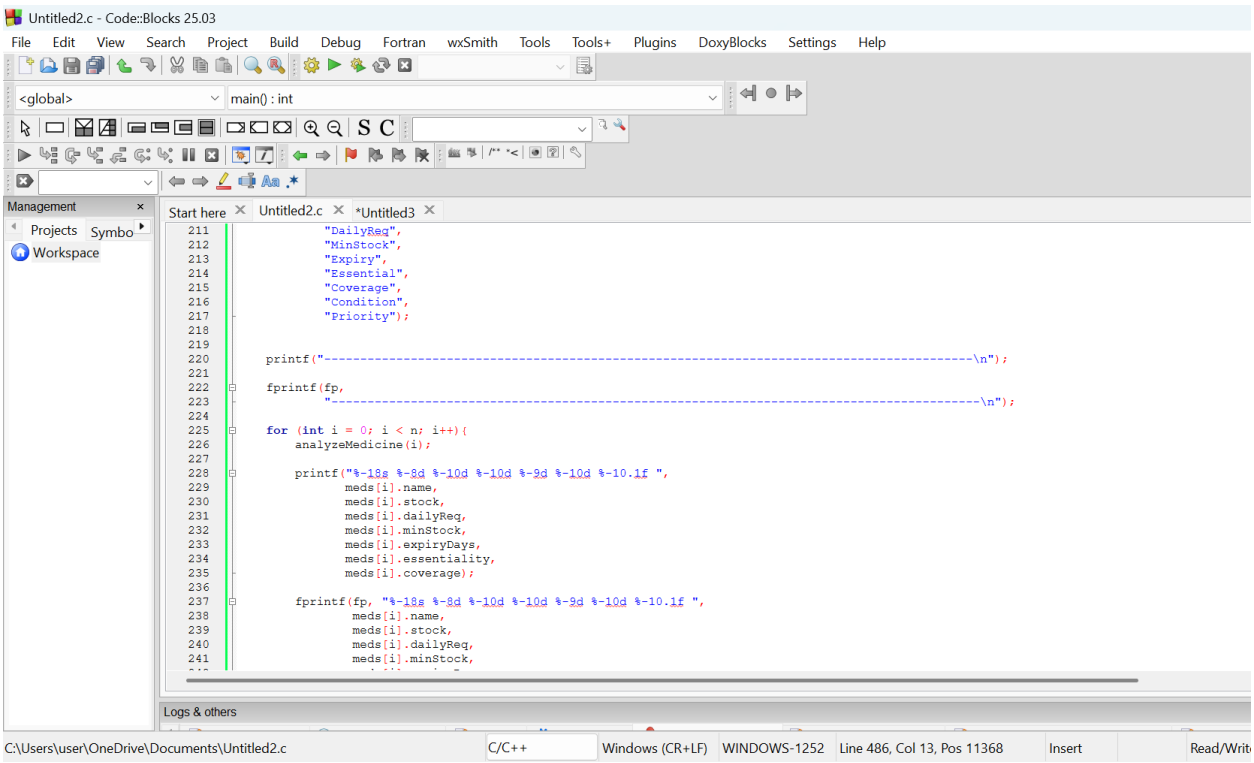

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
61 {
62 /* Calculate stock coverage */
63
64 if (meds[i].dailyReq > 0){
65 meds[i].coverage =
66 (float)meds[i].stock / meds[i].dailyReq;
67 }
68 else
69 {
70 meds[i].coverage = 0;
71 }
72
73 /*
74 Condition:
75
76 5 = Critical
77 4 = Urgent
78 3 = Expiry
79 2 = Reorder
80 1 = Sufficient
81 */
82
83 /* Shortage + expiry concern */
84
85 if (meds[i].stock < meds[i].minStock && meds[i].expiryDays <= 30){
86 meds[i].condition = 5;
87 }
88 /* Essential medicine with low stock */
89 else if (meds[i].stock < meds[i].minStock && meds[i].essentiality == 3){
90 meds[i].condition = 4;
91 }
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert
Read

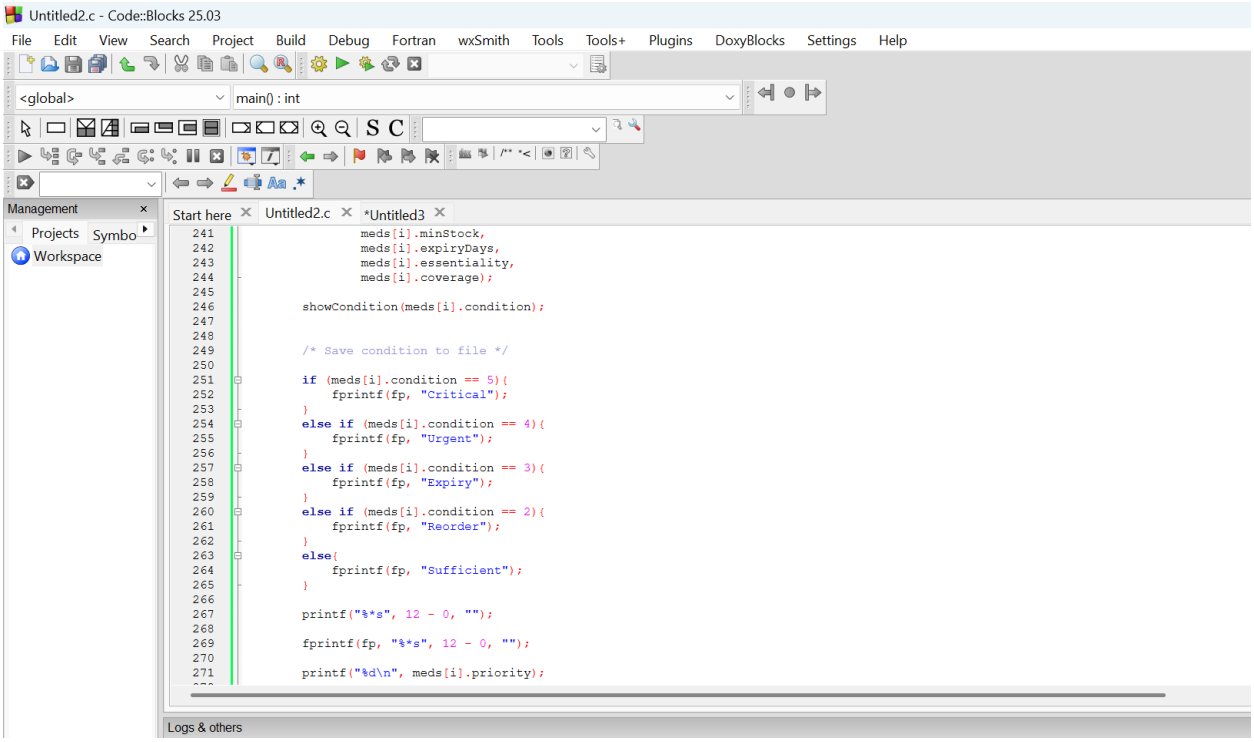

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
91 }
92 /* Approaching expiry */
93 else if (meds[i].expiryDays <= 30){
94 meds[i].condition = 3;
95 }
96 /* Low stock or low coverage */
97 else if (meds[i].stock < meds[i].minStock ||
98 meds[i].coverage < 10 ||
99 meds[i].stock <= meds[i].minStock * 1.2)
100 {
101 meds[i].condition = 2;
102 }
103
104 /* Sufficient stock */
105
106 else
107 {
108 meds[i].condition = 1;
109 }
110 /* Calculate priority */
111 if (meds[i].condition == 5){
112 meds[i].priority = 50;
113 }
114 else if (meds[i].condition == 4)
115 {
116 meds[i].priority = 40;
117 }
118 else if (meds[i].condition == 3)
119 {
120 meds[i].priority = 30;
121 }
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert

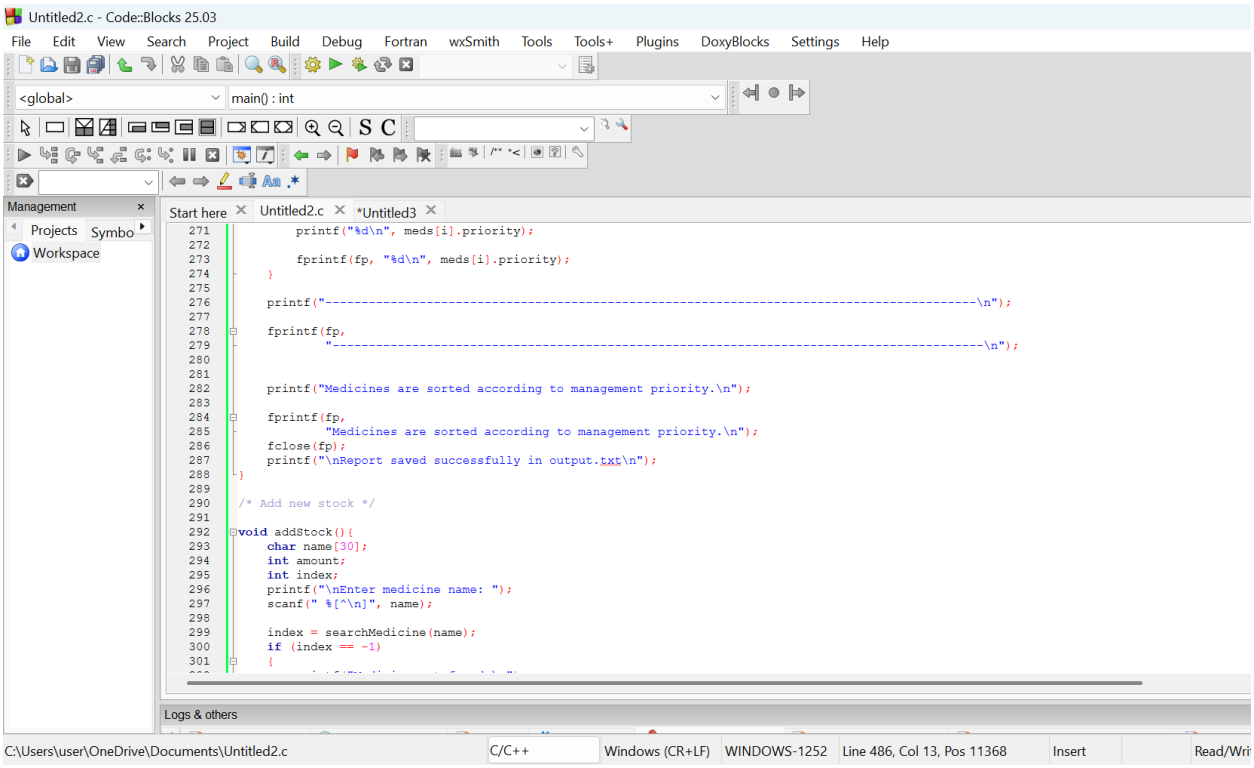

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
121 }
122 else if (meds[i].condition == 2)
123 {
124 meds[i].priority = 20;
125 }
126 else
127 {
128 meds[i].priority = 10;
129 }
130
131 meds[i].priority = meds[i].priority + meds[i].essentiality * 10 - (int)meds[i].coverage;
132 }
133
134 /* Display condition */
135 void showCondition(int condition)
136 {
137 if (condition == 5){
138 printf("Critical");
139 }
140 else if (condition == 4){
141 printf("Urgent");
142 }
143 else if (condition == 3){
144 printf("Expiry");
145 }
146 else if (condition == 2){
147 printf("Reorder");
148 }
149 else{
150 printf("Sufficient");
151 }
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert
Read/Wr

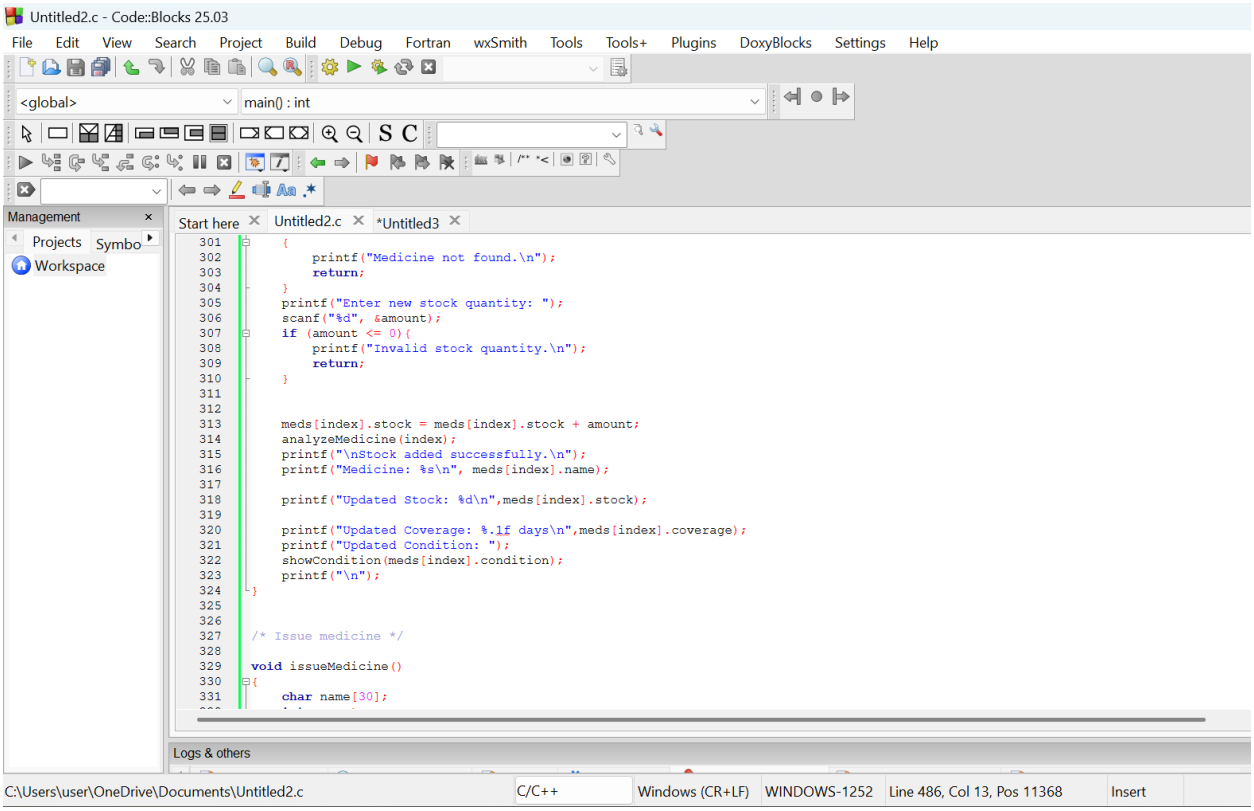

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
151 }
152 }
153
154 /* Sort medicines according to priority */
155 void sortMedicines()
156 {
157 for (int i = 0; i < n - 1; i++){
158 for (int j = 0; j < n - i - 1; j++){
159 analyzeMedicine(j);
160 analyzeMedicine(j + 1);
161
162 if (meds[j].priority < meds[j + 1].priority){
163 struct Medicine temp;
164
165 temp = meds[j];
166 meds[j] = meds[j + 1];
167 meds[j + 1] = temp;
168 }
169 }
170 }
171 }
172
173 /*
174 Show complete medicine report.
175
176 The report is saved in output.txt.
177 The same report is also shown on the screen.
178 */
179
180 void showReport()
181 {
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert
Read/Wri

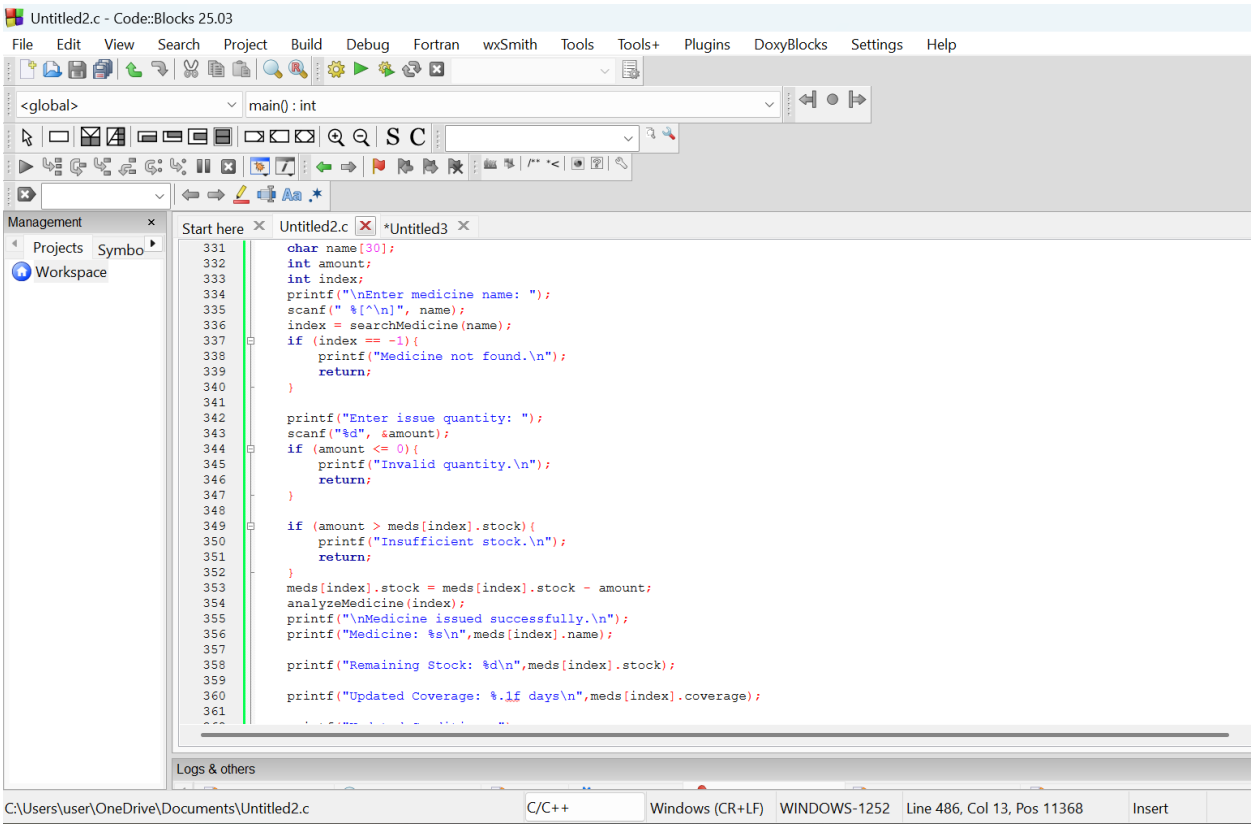

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
181 {
182 FILE *fp = fopen("output.txt", "w");
183
184 if (fp == NULL){
185 printf("\nError: Could not create output.txt\n");
186 return;
187 }
188 sortMedicines();
189
190 printf("\n");
191 printf("MEDICINE STOCK MANAGEMENT REPORT\n");
192 printf("\n");
193 fprintf(fp, "MEDICINE STOCK MANAGEMENT REPORT\n\n");
194
195
196 printf("%-18s %-8s %-10s %-10s %-9s %-10s %-10s %-12s %-8s\n",
197 "Medicine",
198 "Stock",
199 "DailyReq",
200 "MinStock",
201 "Expiry",
202 "Essential",
203 "Coverage",
204 "Condition",
205 "Priority");
206
207
208 fprintf(fp, "%-18s %-8s %-10s %-10s %-9s %-10s %-10s %-12s %-8s\n",
209 "Medicine",
210 "Stock",
211 "DailyReq",
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert
Read/Write

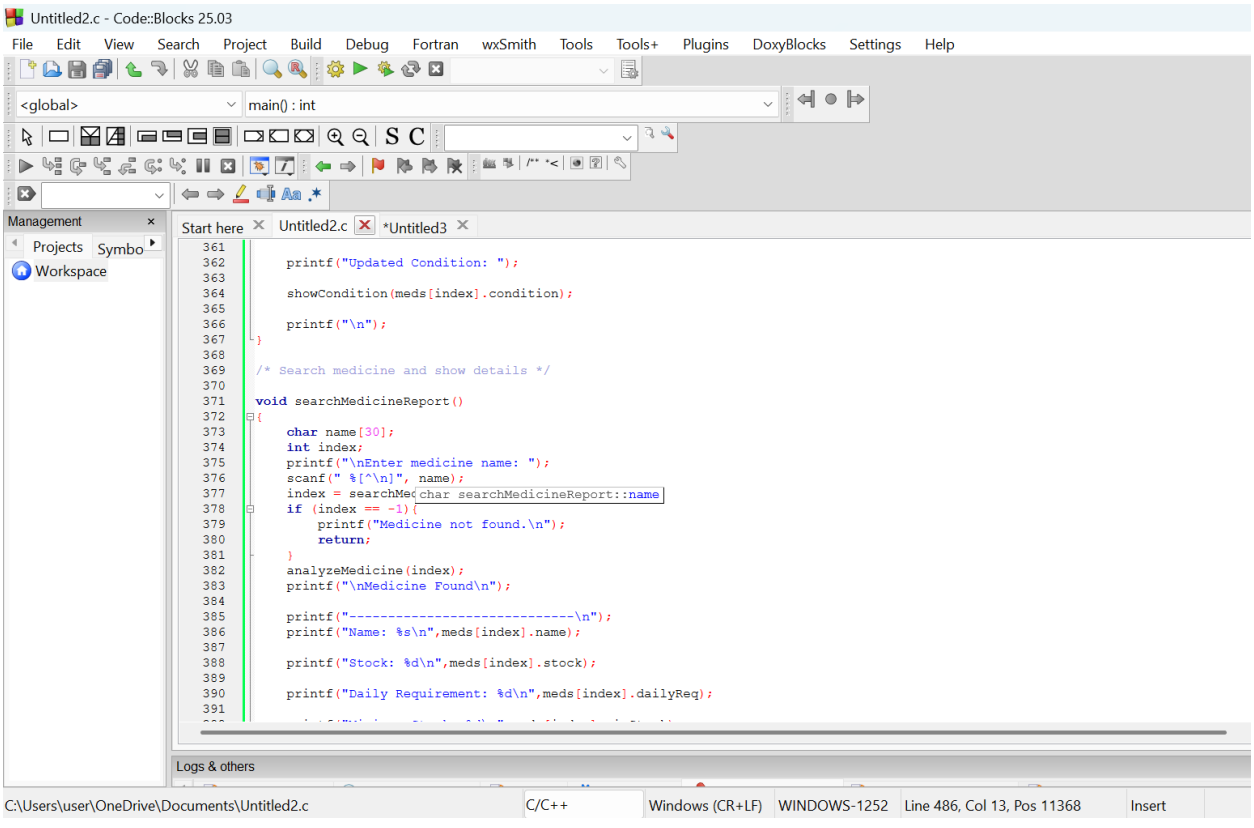

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
"DailyReq",
"MinStock",
"Expiry",
"Essential",
"Coverage",
"Condition",
"Priority");
printf("----------\n");
fprintf(fp,
"----------\n");
for (int i = 0; i < n; i++){
analyzeMedicine(i);
printf("%-18s %-8d %-10d %-10d %-9d %-10d %-10.1f ",
meds[i].name,
meds[i].stock,
meds[i].dailyReq,
meds[i].minStock,
meds[i].expiryDays,
meds[i].essentiality,
meds[i].coverage);
fprintf(fp, "%-18s %-8d %-10d %-10d %-9d %-10d %-10.1f ",
meds[i].name,
meds[i].stock,
meds[i].dailyReq,
meds[i].minStock,
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert
Read/Writ

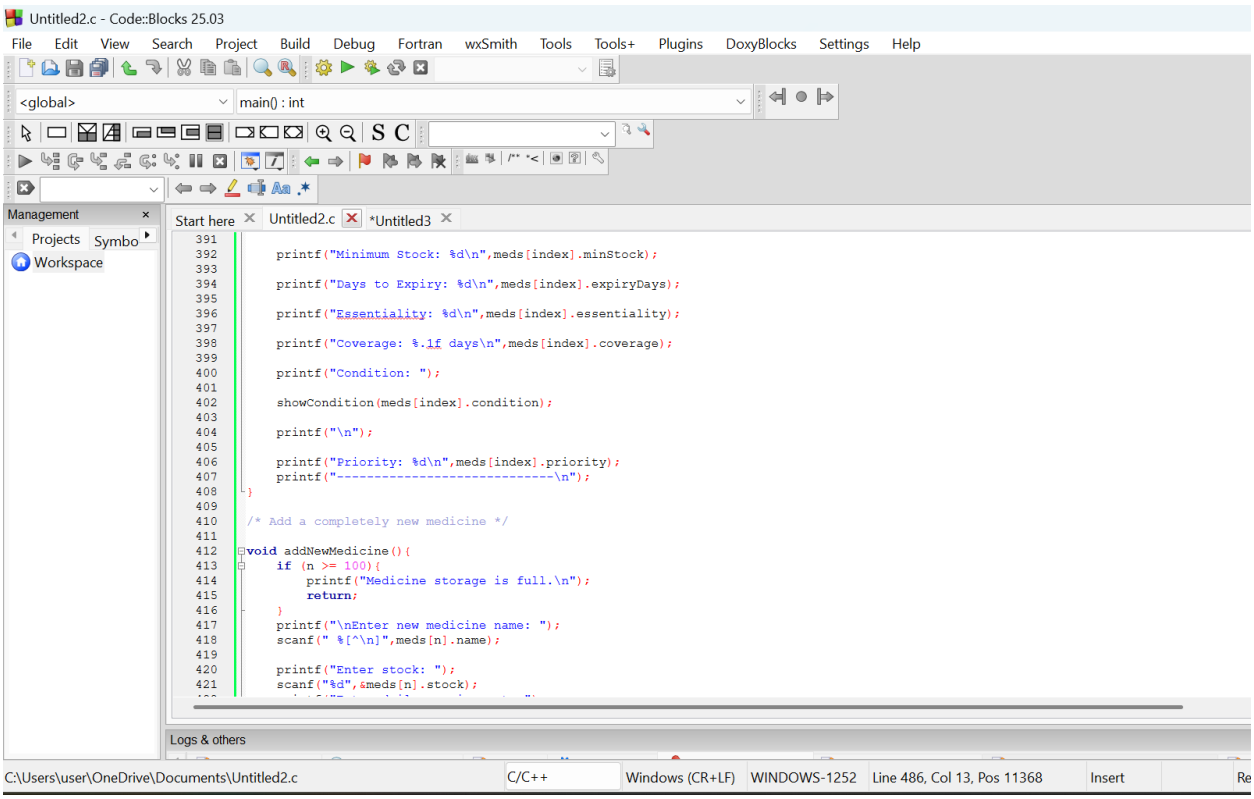

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
meds[i].minStock,
meds[i].expiryDays,
meds[i].essentiality,
meds[i].coverage);
showCondition(meds[i].condition);
/* Save condition to file */
if (meds[i].condition == 5){
fprintf(fp, "Critical");
}
else if (meds[i].condition == 4){
fprintf(fp, "Urgent");
}
else if (meds[i].condition == 3){
fprintf(fp, "Expiry");
}
else if (meds[i].condition == 2){
fprintf(fp, "Reorder");
}
else{
fprintf(fp, "Sufficient");
}
printf("%*s", 12 - 0, "");
fprintf(fp, "%*s", 12 - 0, "");
printf("%d\n", meds[i].priority);
Logs & others

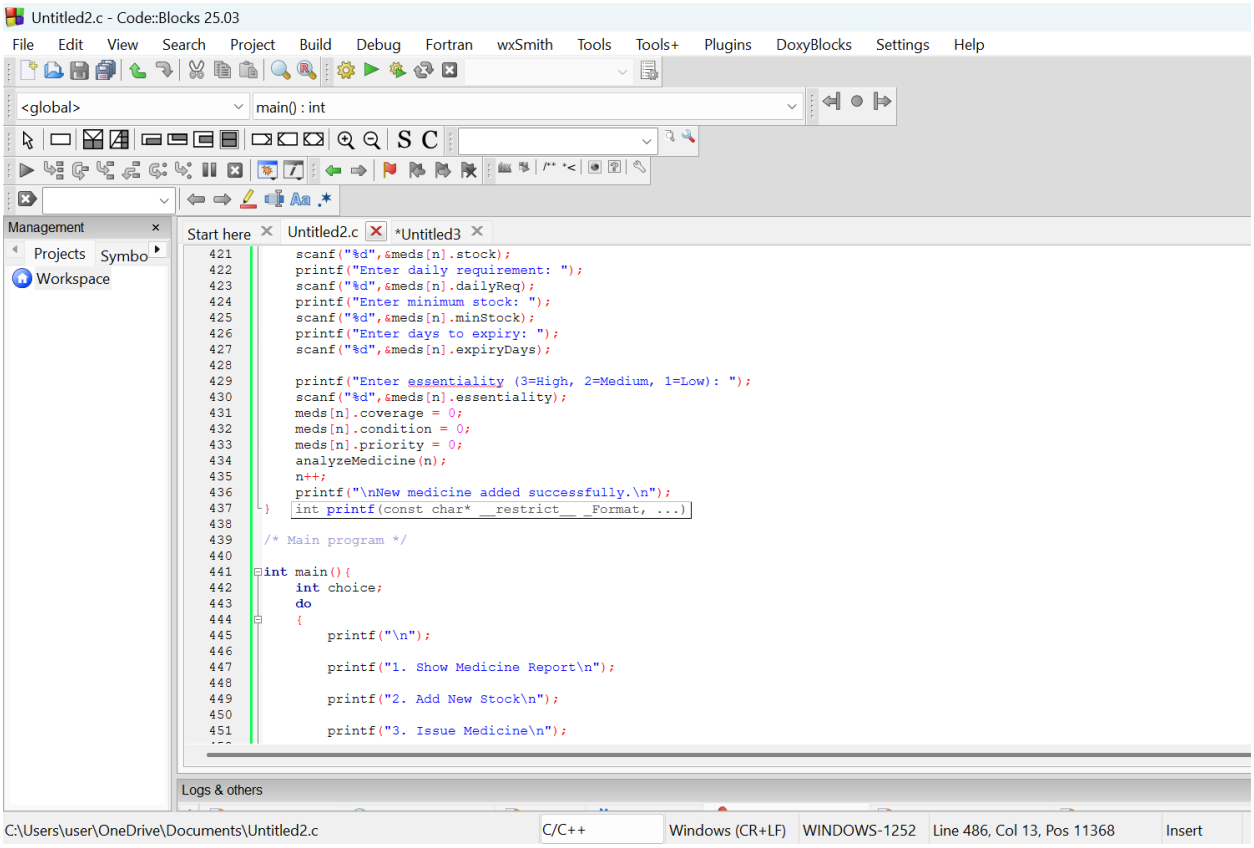

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
271 printf("%d\n", meds[i].priority);
272
273 fprintf(fp, "%d\n", meds[i].priority);
274 }
275
276 printf("----------\n");
277
278 fprintf(fp,
279 "----------\n");
280
281
282 printf("Medicines are sorted according to management priority.\n");
283
284 fprintf(fp,
285 "Medicines are sorted according to management priority.\n");
286 fclose(fp);
287 printf("\nReport saved successfully in output.txt\n");
288 }
289
290 /* Add new stock */
291
292 void addStock(){
293 char name[30];
294 int amount;
295 int index;
296 printf("\nEnter medicine name: ");
297 scanf(" %[^\n]", name);
298
299 index = searchMedicine(name);
300 if (index == -1)
301 {
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert
Read/Wri

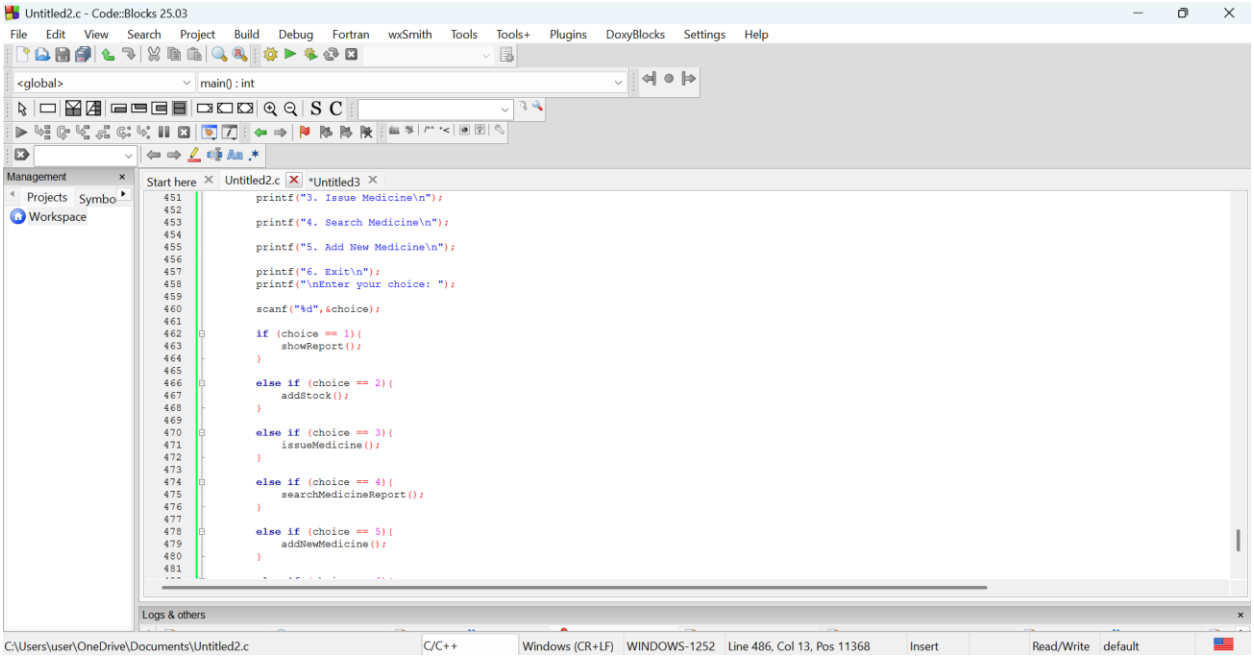

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
301 {
302 printf("Medicine not found.\n");
303 return;
304 }
305 printf("Enter new stock quantity: ");
306 scanf("%d", &amount);
307 if (amount <= 0){
308 printf("Invalid stock quantity.\n");
309 return;
310 }
311
312
313 meds[index].stock = meds[index].stock + amount;
314 analyzeMedicine(index);
315 printf("\nStock added successfully.\n");
316 printf("Medicine: %s\n", meds[index].name);
317
318 printf("Updated Stock: %d\n",meds[index].stock);
319
320 printf("Updated Coverage: %.1f days\n",meds[index].coverage);
321 printf("Updated Condition: ");
322 showCondition(meds[index].condition);
323 printf("\n");
324 }
325
326
327 /* Issue medicine */
328
329 void issueMedicine()
330 {
331 char name[30];
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert

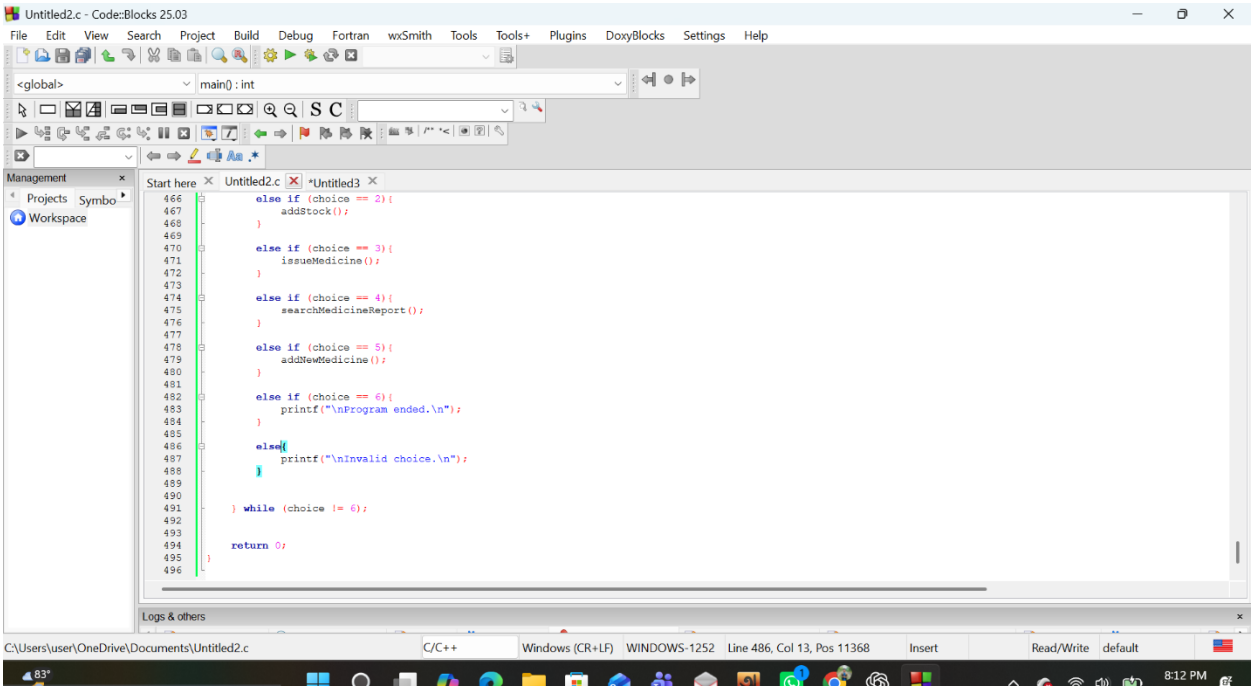

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
331 char name[30];
332 int amount;
333 int index;
334 printf("\nEnter medicine name: ");
335 scanf(" %[^\n]", name);
336 index = searchMedicine(name);
337 if (index == -1){
338 printf("Medicine not found.\n");
339 return;
340 }
341
342 printf("Enter issue quantity: ");
343 scanf("%d", &amount);
344 if (amount <= 0){
345 printf("Invalid quantity.\n");
346 return;
347 }
348
349 if (amount > meds[index].stock){
350 printf("Insufficient stock.\n");
351 return;
352 }
353 meds[index].stock = meds[index].stock - amount;
354 analyzeMedicine(index);
355 printf("\nMedicine issued successfully.\n");
356 printf("Medicine: %s\n",meds[index].name);
357
358 printf("Remaining Stock: %d\n",meds[index].stock);
359
360 printf("Updated Coverage: %.1f days\n",meds[index].coverage);
361
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert


Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects
Symbo
Workspace
Start here
Untitled2.c
*Untitled3
361
362 printf("Updated Condition: ");
363
364 showCondition(meds[index].condition);
365
366 printf("\n");
367 }
368
369 /* Search medicine and show details */
370
371 void searchMedicineReport()
372 {
373 char name[30];
374 int index;
375 printf("\nEnter medicine name: ");
376 scanf(" %[^\n]", name);
377 index = searchMe char searchMedicineReport::name
378 if (index == -1){
379 printf("Medicine not found.\n");
380 return;
381 }
382 analyzeMedicine(index);
383 printf("\nMedicine Found\n");
384
385 printf("----------------------------\n");
386 printf("Name: %s\n",meds[index].name);
387
388 printf("Stock: %d\n",meds[index].stock);
389
390 printf("Daily Requirement: %d\n",meds[index].dailyReq);
391
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert


Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects Symbo
Workspace
Start here
Untitled2.c
*Untitled3
391 printf("Minimum Stock: %d\n",meds[index].minStock);
392
393 printf("Days to Expiry: %d\n",meds[index].expiryDays);
394
395 printf("Essentiality: %d\n",meds[index].essentiality);
396
397 printf("Coverage: %.1f days\n",meds[index].coverage);
398
399 printf("Condition: ");
400
401 showCondition(meds[index].condition);
402
403 printf("\n");
404
405 printf("Priority: %d\n",meds[index].priority);
406 printf("----------------------------\n");
407 }
408
409 /* Add a completely new medicine */
410
411 void addNewMedicine(){
412 if (n >= 100){
413 printf("Medicine storage is full.\n");
414 return;
415 }
416 printf("\nEnter new medicine name: ");
417 scanf(" %[^\n]",meds[n].name);
418
419 printf("Enter stock: ");
420 scanf("%d",&meds[n].stock);
421
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert


Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects Symbo
Workspace
Start here
Untitled2.c
*Untitled3
421 scanf("%d",&meds[n].stock);
422 printf("Enter daily requirement: ");
423 scanf("%d",&meds[n].dailyReq);
424 printf("Enter minimum stock: ");
425 scanf("%d",&meds[n].minStock);
426 printf("Enter days to expiry: ");
427 scanf("%d",&meds[n].expiryDays);
428
429 printf("Enter essentiality (3=High, 2=Medium, 1=Low): ");
430 scanf("%d",&meds[n].essentiality);
431 meds[n].coverage = 0;
432 meds[n].condition = 0;
433 meds[n].priority = 0;
434 analyzeMedicine(n);
435 n++;
436 printf("\nNew medicine added successfully.\n");
437 } int printf(const char* __restrict__ _Format, ...)
438
439 /* Main program */
440
441 int main(){
442 int choice;
443 do
444 {
445 printf("\n");
446
447 printf("1. Show Medicine Report\n");
448
449 printf("2. Add New Stock\n");
450
451 printf("3. Issue Medicine\n");
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert

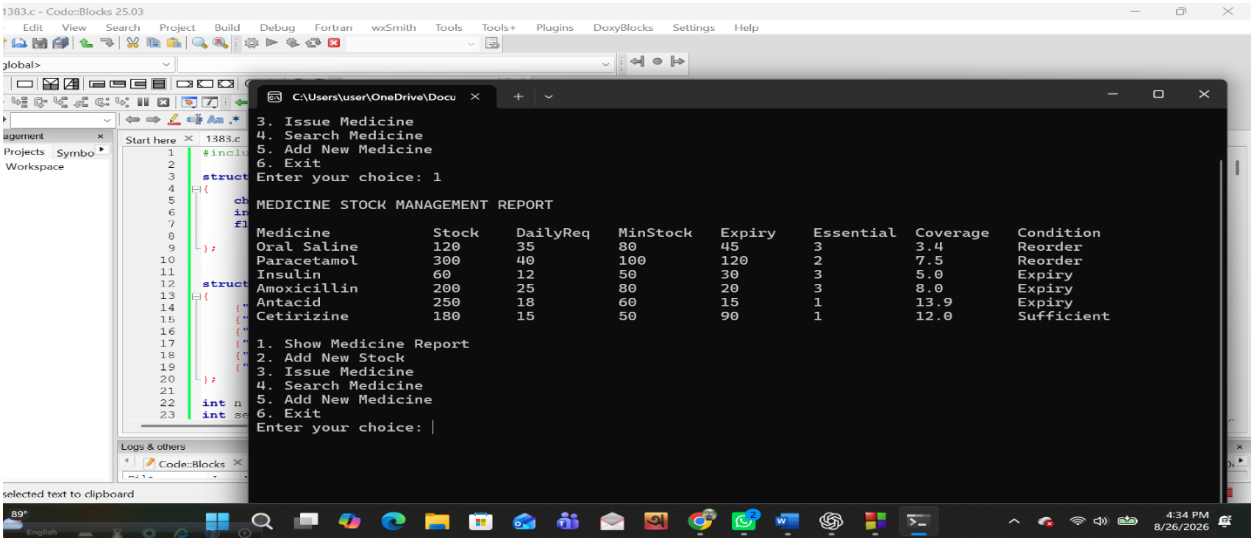

Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects Symbo
Workspace
Start here
Untitled2.c
*Untitled3
printf("3. Issue Medicine\n");
printf("4. Search Medicine\n");
printf("5. Add New Medicine\n");
printf("6. Exit\n");
printf("\nEnter your choice: ");
scanf("%d",&choice);
if (choice == 1){
showReport();
}
else if (choice == 2){
addStock();
}
else if (choice == 3){
issueMedicine();
}
else if (choice == 4){
searchMedicineReport();
}
else if (choice == 5){
addNewMedicine();
}
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert
Read/Write
default


Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global>
main() : int
Management
Projects Symbo
Workspace
Start here
Untitled2.c
*Untitled3
else if (choice == 2){
addStock();
}
else if (choice == 3){
issueMedicine();
}
else if (choice == 4){
searchMedicineReport();
}
else if (choice == 5){
addNewMedicine();
}
else if (choice == 6){
printf("\nProgram ended.\n");
}
else{
printf("\nInvalid choice.\n");
}
} while (choice != 6);
return 0;
}
Logs & others
C:\Users\user\OneDrive\Documents\Untitled2.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 486, Col 13, Pos 11368
Insert
Read/Write
default
83°
8:12 PM

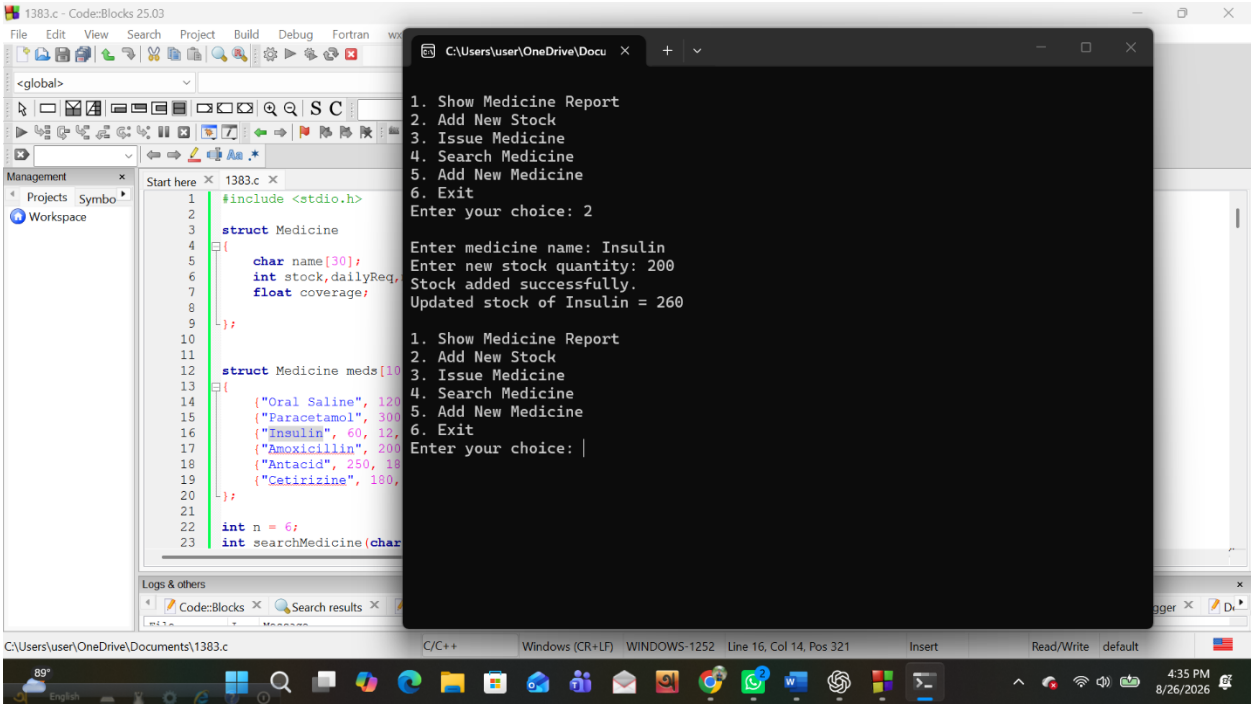
7.Testing and Validation

Program run

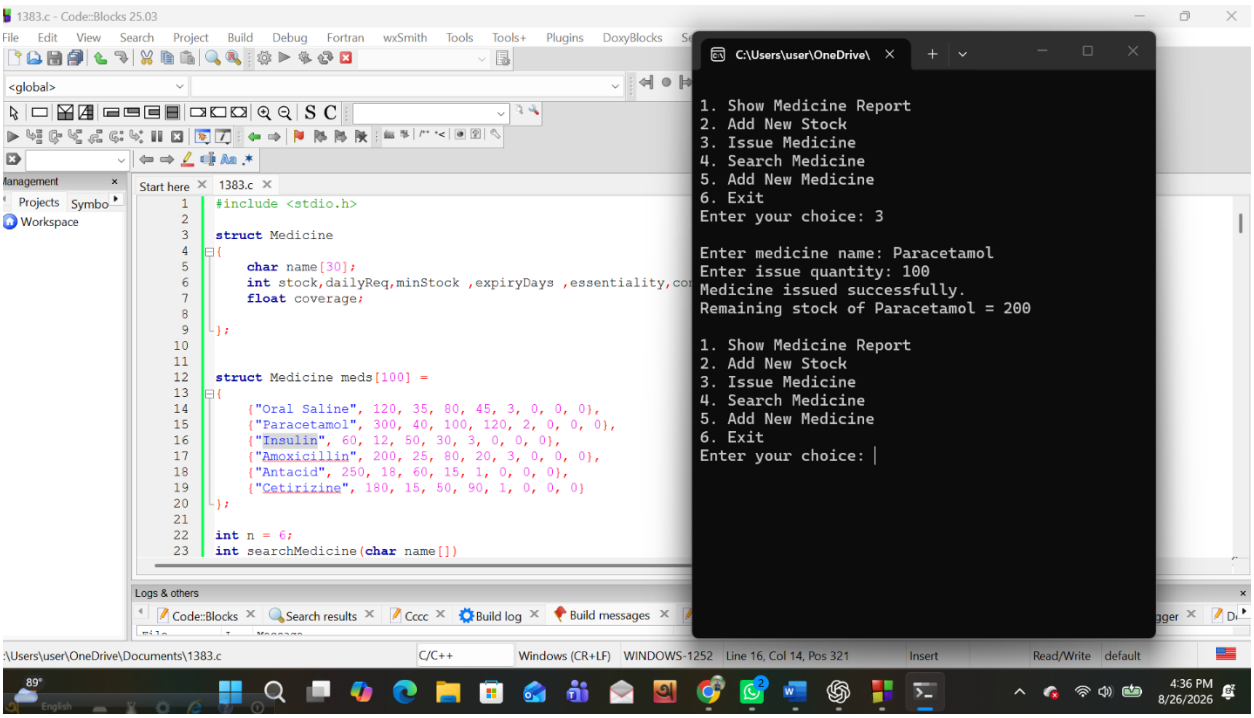
Case 1



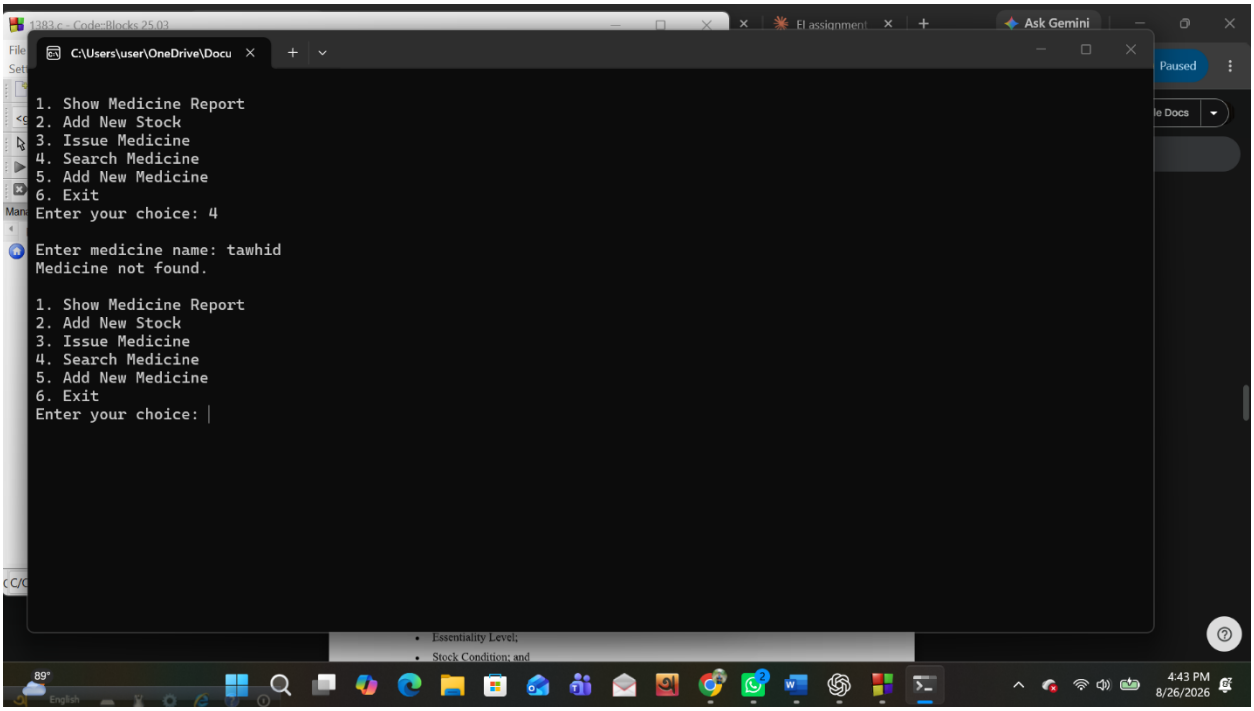
Case 2



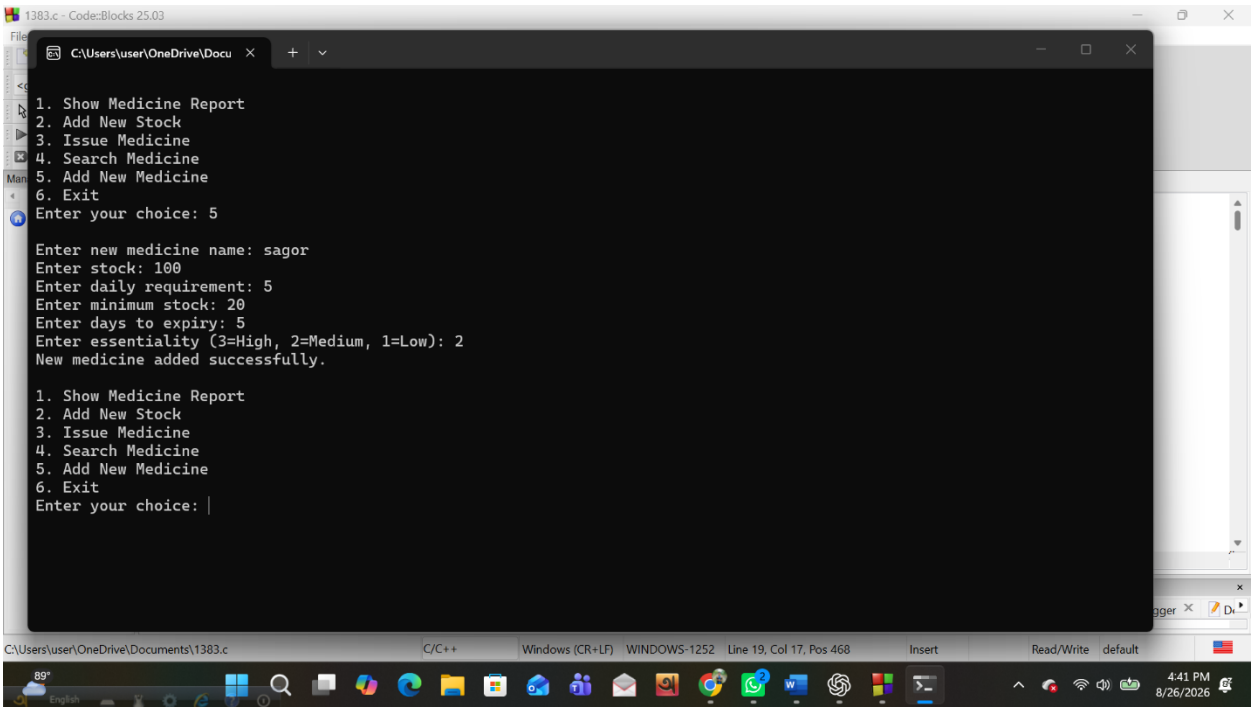
Case 3


1383.c - Code::Blocks 25.03
C:\Users\user\OneDrive\
1. Show Medicine Report
2. Add New Stock
3. Issue Medicine
4. Search Medicine
5. Add New Medicine
6. Exit
Enter your choice: 3
Enter medicine name: Paracetamol
Enter issue quantity: 100
Medicine issued successfully.
Remaining stock of Paracetamol = 200
1. Show Medicine Report
2. Add New Stock
3. Issue Medicine
4. Search Medicine
5. Add New Medicine
6. Exit
Enter your choice:
#include <stdio.h>
struct Medicine
char name[30];
int stock,dailyReq,minStock ,expiryDays ,essentiality,co
float coverage;
struct Medicine meds[100] =
{"Oral Saline", 120, 35, 80, 45, 3, 0, 0, 0},
{"Paracetamol", 300, 40, 100, 120, 2, 0, 0, 0},
{"Insulin", 60, 12, 50, 30, 3, 0, 0, 0},
{"Amoxicillin", 200, 25, 80, 20, 3, 0, 0, 0},
{"Antacid", 250, 18, 60, 15, 1, 0, 0, 0},
{"Cetirizine", 180, 15, 50, 90, 1, 0, 0, 0}
int n = 6;
int searchMedicine(char name[])
Logs & others
Code::Blocks
Search results
Build log
Build messages
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 16, Col 14, Pos 321
Insert
Read/Write
default
4:36 PM
8/26/2026

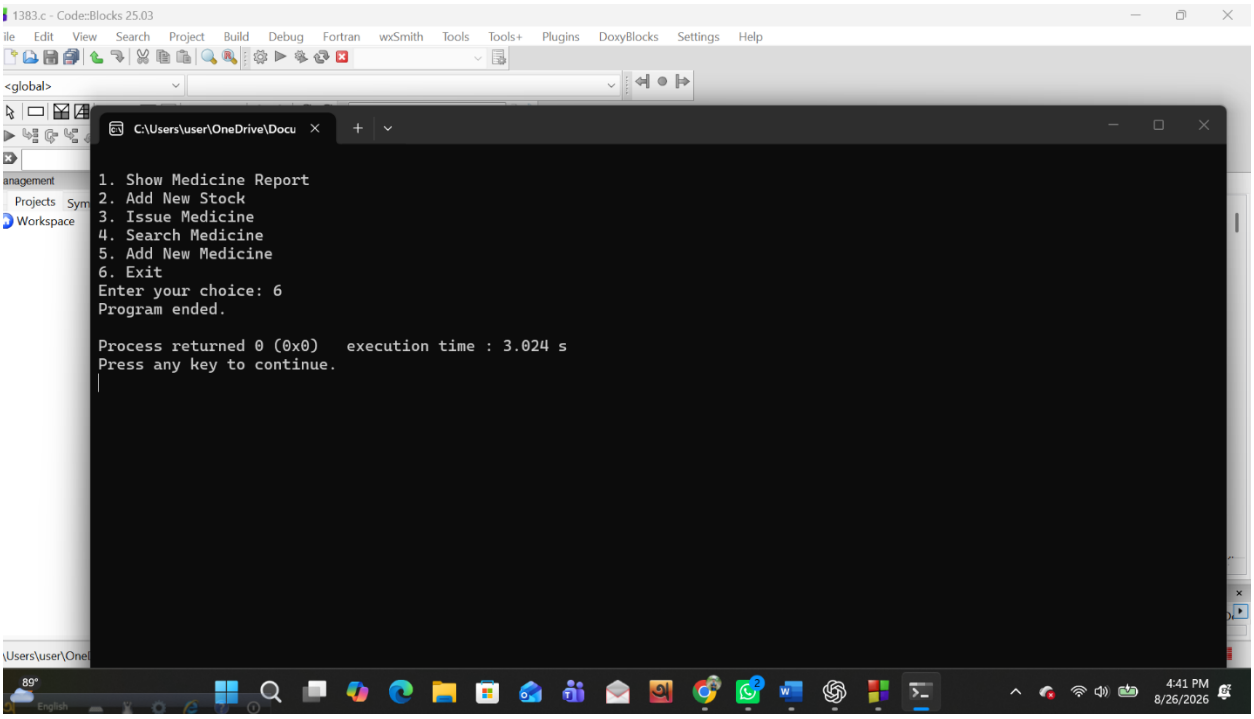
Case 4


C:\Users\user\OneDrive\Docu
1. Show Medicine Report
2. Add New Stock
3. Issue Medicine
4. Search Medicine
5. Add New Medicine
6. Exit
Enter your choice: 4
Enter medicine name: tawhid
Medicine not found.
1. Show Medicine Report
2. Add New Stock
3. Issue Medicine
4. Search Medicine
5. Add New Medicine
6. Exit
Enter your choice:
Ask Gemini
Paused
Essentiality Level;
Stock Condition; and
4:43 PM
8/26/2026

Case 5


1383.c - Code::Blocks 25.03
C:\Users\user\OneDrive\Docu
1. Show Medicine Report
2. Add New Stock
3. Issue Medicine
4. Search Medicine
5. Add New Medicine
6. Exit
Enter your choice: 5
Enter new medicine name: sagor
Enter stock: 100
Enter daily requirement: 5
Enter minimum stock: 20
Enter days to expiry: 5
Enter essentiality (3=High, 2=Medium, 1=Low): 2
New medicine added successfully.
1. Show Medicine Report
2. Add New Stock
3. Issue Medicine
4. Search Medicine
5. Add New Medicine
6. Exit
Enter your choice:
C:\Users\user\OneDrive\Documents\1383.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 19, Col 17, Pos 468
Insert
Read/Write
default
4:41 PM
8/26/2026

Case 6


1383.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
C:\Users\user\OneDrive\Docu
1. Show Medicine Report
2. Add New Stock
3. Issue Medicine
4. Search Medicine
5. Add New Medicine
6. Exit
Enter your choice: 6
Program ended.
Process returned 0 (0x0) execution time : 3.024 s
Press any key to continue.
4:41 PM
8/26/2026

8. Limitations

The proposed solution rests on a number of assumptions and carries corresponding limitations that should be understood when interpreting its output.

- The average daily requirement recorded for each medicine is treated as a stable, representative rate of consumption; the program does not account for seasonal variation, disease outbreaks, or other irregular surges in patient demand that could alter this rate over short periods.
- The days remaining to expiry is treated as applying uniformly to the entire available stock of a medicine, since the supplied dataset does not distinguish between separate batches of the same medicine that may carry different expiry periods; a health center holding multiple batches with different expiry dates would need to extend the data organization described in Section 5.1 to a batch-level record.
- The minimum stock level and the essentiality level are treated as fixed reference values supplied by the center's management; the program does not itself determine what these values ought to be, nor does it adjust them automatically over time.
- The dataset size assumed for demonstration is small, and the sorting technique selected in Section 6 is chosen accordingly; a very much larger medicine inventory would warrant reconsideration of the selected technique.
- The classification and priority methods described in Section 4 are one reasonable, explained approach among several that could satisfy the assignment brief, which explicitly permits students to adopt a different, justified classification; another reasonable method would be expected to produce broadly similar, though not necessarily identical, orderings on the supplied dataset.

9. Conclusion

This assignment required the design of a structured, C-language solution to a realistic inventory-management problem in which shortage risk and expiry risk must be considered together, rather than in isolation, and in which not every medicine carries the same clinical weight. The proposed solution derives a stock-coverage period and an expiry-exposure quantity for every medicine from the supplied data, uses these derived quantities together with the essentiality level to classify each medicine into one of five stock conditions, computes a combined priority indicator to order the medicines by the urgency of managerial attention they require, and resolves equal-priority cases through a short, reasoned sequence of secondary comparisons. The design keeps every derived value attached to the medicine record it describes, recalculates only what genuinely changes after a stock update, and is intended to remain correct when the supplied values, or the number of medicines, are changed, in line with the requirements stated in the assignment brief. Sections 7 and 8 of this report, concerning the implementation and its testing, are to be completed separately alongside the submitted source code and test evidence.

10. References

[1] Course materials and assignment brief, "Assignment on Medicine Stock and Expiry Management," CSC 1383 Structured Programming, Summer 2026.